

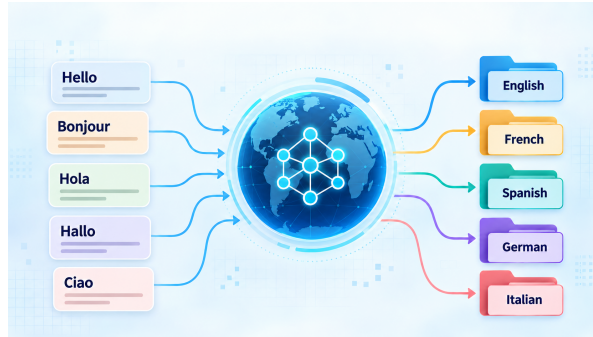
Large Language Models (Homework 1)

Due date : 2026/10/16 23:55:00 (Hard Deadline)

1 N-Gram, RNN, and LSTM Models for Language Identification (50%)

In this section, you will implement character-level n -gram, recurrent neural network (RNN), and long short-term memory (LSTM) language models for **language identification**. The dataset used in this exercise is the **TEDTalks Dataset for 63 Languages**, which is available on **Kaggle**. This exercise focuses on five languages: **English, French, Spanish, Portuguese, and German**. For each model type, you will construct one language model for each language and predict the language of an input text by selecting the model with the **lowest perplexity**. Using five languages that share the Latin alphabet makes the task focus on language-specific character patterns rather than differences between writing systems.

Dataset description:



- The course-provided dataset contains five languages: **English, French, Spanish, Portuguese, and German**, with **1,400 text instances per language**.
- The dataset has already been split into **train.csv**, **val.csv**, and **test.csv**, with 1120 training, 140 validation, and 140 test instances per language.
- Each file contains the columns **Text** and **Language**.
- Use **train.csv** for training, **val.csv** for validation and model selection, and **test.csv** only for final evaluation. **Do not resample, reshuffle, or repartition the data.**
- Split each text into individual characters and construct a shared character vocabulary from the training data.
- Add **<PAD>**, **<UNK>**, **<BOS>**, and **<EOS>** tokens, and use the **same vocabulary and special-token policy** for all models.

Please follow the steps below to implement your program:

1. Implement character-level **n -gram language models** for language identification, where $C_l(\cdot)$ represents the number of occurrences in language l , and $|\mathcal{V}|$ is the size of the shared character vocabulary. You may use `defaultdict()` in Python for implementation. For each language, construct the following bigram and trigram models using **Add- α smoothing with $\alpha = 1$** :

$$p(c_t \mid c_{t-1}, l) = \frac{C_l(c_{t-1}, c_t) + \alpha}{C_l(c_{t-1}) + \alpha|\mathcal{V}|},$$
$$p(c_t \mid c_{t-2}, c_{t-1}, l) = \frac{C_l(c_{t-2}, c_{t-1}, c_t) + \alpha}{C_l(c_{t-2}, c_{t-1}) + \alpha|\mathcal{V}|}.$$

Use $\alpha = 1$ for all languages and both n -gram settings. This smoothing prevents unseen character sequences from producing zero probability. For a text $x = (c_1, \dots, c_T)$, calculate the average negative log-likelihood and perplexity under each language model, and choose the language l with the lowest perplexity:

$$\begin{aligned}\text{NLL}(x | l) &= -\frac{1}{T} \sum_{t=1}^T \log p(c_t | c_1, \dots, c_{t-1}, l), \\ \text{PPL}(x | l) &= \exp(\text{NLL}(x | l)), \\ \mathcal{L} &= \{\text{English, French, Spanish, Portuguese, German}\}, \\ \hat{l} &= \arg \min_{l \in \mathcal{L}} \text{PPL}(x | l).\end{aligned}$$

For validation and test evaluation, corpus-level perplexity is defined as

$$\text{PPL}_{\text{corpus}} = \exp \left(\frac{\sum_{i=1}^N \sum_{t=1}^{T_i} -\log p(c_{i,t} | c_{i,<t}, l_i)}{\sum_{i=1}^N T_i} \right),$$

where l_i is the ground-truth language of the i -th text and T_i is the number of valid prediction targets.

- (a) Use $n = 2$ and $n = 3$ to build five bigram and five trigram models using `train.csv` and Add- α smoothing with $\alpha = 1$. Evaluate their corpus-level perplexity on `val.csv`. (5%)
 - (b) Evaluate both models on `test.csv` and report Accuracy, Macro-F1, corpus-level perplexity, and the confusion matrix. (5%)
 - (c) Measure model construction time, inference time, and peak memory usage, and analyze representative incorrect predictions. Discuss which languages are easily confused and whether shorter texts are more difficult to identify. (5%)
2. You are allowed to use the `PyTorch library` to implement five character-level **Recurrent Neural Network (RNN)** language models, one for each language. You must create a character vocabulary and implement the conversion between characters and indices. Each model predicts the next character from the preceding characters. Use cross-entropy as the loss function and Adam as the optimizer. The suggested hyperparameter settings are as follows:
- embedding dimension : 128
 - hidden dimension : 128
 - number of layers : 2
 - learning rate : 0.001
 - number of epochs : 10
 - batch size : 32
- (a) Plot the **learning curves (training loss)** and the **validation perplexity curves**. (5%)
 - (b) Use `val.csv` to select model settings and the final checkpoint. Evaluate the selected RNN models on `test.csv` and report Accuracy, Macro-F1, and corpus-level perplexity. (5%)
 - (c) Use the trained RNN models to predict the language of every text in `test.csv`. Report the confusion matrix, training time, inference time, and peak memory usage, and discuss representative correct and incorrect predictions. (5%)

3. You are also required to implement five character-level **Long Short-Term Memory (LSTM)** language models, one for each language, as an extension of the RNN models. The models should follow the same settings as the RNN implementation, including the data split, character vocabulary, character-to-index mapping, loss function, optimizer, and suggested hyperparameters.
 - (a) Plot the **learning curves (training loss)** and the **validation perplexity curves**. (5%)
 - (b) Use **val.csv** to select model settings and the final checkpoint. Evaluate the selected LSTM models on **test.csv** and report Accuracy, Macro-F1, and corpus-level perplexity. (5%)
 - (c) Use the trained LSTM models to predict the language of every text in **test.csv**. Report the confusion matrix, training time, inference time, and peak memory usage, and discuss representative correct and incorrect predictions. (5%)
4. Analyze and compare the differences among the bigram, trigram, RNN, and LSTM models, including the hardware resources required for their operation, the computation time taken for training and inference, and their Accuracy, Macro-F1, and perplexity results. Discuss whether lower perplexity always corresponds to higher language identification accuracy, which language pairs are most frequently confused, and whether model performance decreases for shorter texts. (5%)

NOTE:

- Exclude **<BOS>** and **<PAD>** from prediction targets; include **<EOS>**.
- Do not average individual text-level perplexities. All validation and test perplexities must use the corpus-level definition above.

2 Self-Attention Layer for Semantic Similarity Scoring (50%)

In this section, you will participate in a **Kaggle competition** for a semantic similarity regression task based on the **U.S. Patent Phrase to Phrase Matching** dataset. Each sample contains an anchor phrase, a target phrase, and a patent context, and the goal is to predict a similarity score between 0 and 1. You will complete a Transformer encoder using the provided notebook, train and evaluate the model on the provided data, and submit predictions to the course Kaggle competition.

Dataset description:



- The original labeled training data contain 36,473 phrase pairs.
- The course data use **train.csv** with 25,531 entries, **val.csv** with 3,647 entries, and **test.csv** with 7,295 entries.
- Each entry in **train.csv** and **val.csv** contains **id**, **anchor**, **target**, **context**, and **score**. Each entry in **test.csv** contains only **id**, **anchor**, **target**, and **context**; the score is hidden for Kaggle evaluation.

- The `context` field represents the patent classification context in which the semantic relationship between the anchor and target phrases is evaluated.
- The ground-truth `score` takes one of five values, $\{0, 0.25, 0.5, 0.75, 1\}$, and represents semantic similarity within the patent context. The sigmoid function constrains the model output to the interval $(0, 1)$, which is compatible with the target score range $[0, 1]$.
- You must generate predictions for `test.csv` and submit them to the course Kaggle competition.

Please follow the requirements below to implement your program:

1. Construct a single input sequence from the context, anchor, and target fields. Encode the sequence using token embeddings, positional embeddings, and Transformer encoder layers. By default, apply masked mean pooling and a regression head to obtain one similarity score:

$$z = [\text{CONTEXT}] \text{ context } [\text{SEP}] \text{ anchor } [\text{SEP}] \text{ target}$$

$$H = \text{TransformerEncoder}(\text{Embedding}(z) + \text{PositionalEmbedding}(z))$$

$$h_{\text{pool}} = \frac{\sum_i m_i h_i}{\sum_i m_i}$$

$$\hat{s} = \text{sigmoid}(W h_{\text{pool}} + b)$$

Complete all sections marked as “**TODO**” in the provided notebook, including the query, key, and value projections; scaled dot-product attention; head concatenation; residual connections; layer normalization; and the feed-forward network. **Do not use `torch.nn.MultiheadAttention` or a pretrained Transformer model.**

2. Use mean squared error as the training loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (s_i - \hat{s}_i)^2.$$

The model is trained using MSE, while model selection and Kaggle leaderboard ranking are based on **Pearson correlation**.

Use the provided vocabulary construction procedure, maximum sequence length, embedding dimension, number of layers, batch size, and random seed unless the corresponding setting is explicitly listed as tunable.

The embedding dimension must be divisible by the number of attention heads, that is, $d_{\text{model}} \bmod h = 0$. In particular, the embedding dimension must support all required settings $h \in \{1, 4, 8\}$.

3. Complete the following experiments and analyses:
 - (a) **Set the number of attention heads to 4.** Plot the learning curves for training loss and validation loss, as well as the validation Pearson correlation curve. (10%)
 - (b) Evaluate the model on `val.csv` and report Pearson correlation and RMSE. (5%)
 - (c) Set the number of attention heads to **1, 4, and 8** using the same data split and training budget. Compare their validation performance and discuss the results. (5%)
 - (d) Improve validation Pearson correlation by tuning permitted settings, such as hyperparameters, the optimizer, the learning-rate scheduler, or the pooling strategy. **Document every change.** (5%)
 - (e) Use your final model to predict all samples in `test.csv`. Generate `submission.csv` following the format of `sample_submission.csv`, upload it to the course Kaggle competition, and include a screenshot of the final leaderboard score. **Do not generate, renumber, or reorder the IDs.** (25%)

3 Rules

- In your course submission, you need to submit two notebooks. Only the following filename format is accepted:
 - `hw1_<ProblemNumber>_<StudentID>.ipynb`, containing executable code, results, figures, metric values, and written discussion for each exercise (for example, `hw1_2_0123456.ipynb`).
- Implementation will be graded based on
 - Completeness
 - Algorithm correctness
 - Description of model design
 - Experimental analysis and reproducibility
 - Kaggle performance
- Only [Python](#) implementations are accepted.
- For the [n-gram problem](#), you may use only basic libraries such as [NumPy](#), [pandas](#), [collections](#), [math](#), and [random](#). [N-gram libraries and pretrained models are not permitted](#); you must implement the models yourself.
- For the [RNN, LSTM, and self-attention problems](#), you may use [PyTorch](#). However, [torch.nn.MultiheadAttention](#) and pretrained Transformer encoders are not permitted.
- You must use a [GPU](#) for the RNN, LSTM, and Transformer experiments.
- For Problem 2, each student must make at least one valid Kaggle submission and include the final leaderboard score in the notebook.
- Do not use external labeled data unless the assignment announcement explicitly permits it.
- [DO NOT PLAGIARIZE](#). Program similarity and report similarity will be checked.